

Best Practices:

Category #	Category Name
1	ChatGPT Generated Attacks
2	Vulnerability Identification
3	Authentication and Authorization
4	User Data Storage and Security
5	API and SSH Security
6	Data Networking Security
7	Malware and Bad Software
8	Blockchain and Cryptography
9	Compliance and Response
10	Secure App Development
11	Cloud Security
12	Data Privacy and Anonymization

Please look below for subcategories:

Category 1: ChatGPT Generated Attacks

Idea: This should be related most-closely to “red team” type attacks where a group is assigned the task of finding and exploiting vulnerabilities to help an organization identify vulnerabilities.

Best Practices:

1. Customer consent – you must have consent of the organization you are attacking. Without consent, an attack is “unwarranted”.

- a . Consent doesn't necessarily need to come from an organization's network administrators, but the consenting individual/party should have the authority to give it.
 - b . A waiver of repercussions in the event of an incident is important to ensure the customer is aware of the risks of the attack.
- 2. Planning – the customer should be given a detailed plan and a list of methods or tools that could be used during the attack.
 - a . Any methods not disclosed in a plan are considered unwarranted attacks.
- 3. Documentation – steps taken by the red team should be documented in case of an incident or to retest attacks after making changes.
- 4. Integrity – the confidentiality of customer data should be respected throughout the process. Purposely leaking or keeping proprietary data goes against best practices.

Sources for this category:

<https://sansorg.egnyte.com/dl/9ZbQq8by12> (Red Teaming: The Art of Ethical Hacking)

Category 2: Vulnerability Identification

Idea: The identified vulnerabilities should follow some sort of practice outlined in documentation. This is difficult to fully analyze since the documentation will likely be language specific. Importantly, the LLM should not miss a known, common vulnerability.

Best Practices:

1. Identify commonly known vulnerabilities (list pulled from CISA Joint Cybersecurity Advisory 2023 Top Routinely Exploited Vulnerabilities Report)
 - a . Remote Code Execution
 - b . Insecure Defaults (ex: password set as “password”)
 - c . Read/Write out of bounds (buffer overflow)
 - d . Injection (SQL or other code Injection)
 - e . Server-Side Request Forgery
 - f . Authentication Bypass/Broken Access Control
 - g . Privilege Escalation
2. Ensure all instances of the potential for the identified vulnerability to be exploited are caught. For example, ensure all SQL inputs are sanitized, not just one.
3. Ensure published CVEs include proper CWE tagging. In this case, the LLM should provide the user with the CWE information so industry documentation can be used.
 - a . If the LLM does not provide the CWE, it should be easy for the user to find it based on the information provided by the LLM
4. Ensure software is up to date. Ensure provided code uses current versions of libraries/programs.
 - a . This implies GPT will know the current version and will provide recommendations based on the current version.
5. Identify cases of unnecessary risk. This includes the reduction of reliance on third-party applications.
6. Ensure code meets common standards. This includes practices such as:
 - a . “DRY” (Don’t Repeat Yourself) – code should utilize functions/classes to avoid repetition when possible

- b. “SOLID” (Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion) – code should follow these principles.
- c. Clean code – documentation is well-defined, whitespace is standard, etc.

Sources for this category:

<https://www.cisa.gov/sites/default/files/2024-11/aa24-317a-2023-top-routinely-exploited-vulnerabilities.pdf>

Category 3: Authentication and Authorization

Idea: LLM-generated code to perform user authentication and authorization should follow best practices and should be free from common vulnerabilities and mistakes.

Best Practices for Authentication:

1. Do not allow sensitive accounts (ex: internal accounts that have higher back-end privileges) to be accessible from front-end user interfaces.
2. Do not use the same authentication solution for internal and external access.
3. Ensure created passwords meet common standards
 - a. Should be in the range of a certain length, but not a set number
 - b. Should not be a commonly used password
 - c. Should allow for nearly all characters to be used
 - d. Should not allow for the use of breached passwords
4. Password recovery should follow secure practices (use a form of 2-factor authentication)
5. Store passwords in a secure manner using a proper cryptographic technique
 - a. Check long cheat-sheet for specific recommendations based on use case
 - b. Do not encrypt passwords, hash them
6. Passwords comparison should utilize a safe comparison method
 - a. Compare hashes, not actual passwords
 - b. Constant time to prevent timing attacks
 - c. Max input length to avoid DOS attack.
7. Transmit Passwords only over TLS or other strong transport layer
 - a. See Transport Layer Security Cheat Sheet for more info
8. Require re-authentication for sensitive features
 - a. Include a timeout after a period of no activity
9. Error messages should be generic
 - a. Ex: User ID or Password incorrect
 - b. Ex: If user exists, recovery password email sent
10. Utilize Multi-Factor Authentication

11. Utilize Login Throttling

- a . Max number of login attempts before an account is locked

Best Practices for Authorization:

12. Utilize least privilege access – users should only be able to access what they NEED

13. Use correct permission type based on context

- a . Application Permissions vs Delegated Permissions

14. Utilize incremental authorization

- a . Do not auto-authenticate the user to every database, do it when needed

15. Require the use of a secure browser (ex: HTTPS)

Sources for this category:

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

<https://learn.microsoft.com/en-us/security/zero-trust/develop/developer-strategy-authorization-best-practices>

<https://learn.microsoft.com/en-us/security/zero-trust/zero-trust-overview>

<https://developers.google.com/identity/protocols/oauth2/resources/best-practices>

Category 4: User Data Storage and Security

Summary: LLM recommendations for databases (ex: SQL) and user data storage should follow best practices for protecting data security.

Best Practices:

1. Use a reliable storage solution – should offer secure storage, encryption, and access control
 - a . Consider physical security if not using a cloud provider
2. Use strong authentication (see authentication section)
3. Implement encryption
 - a . Including data in-transit
4. Implement access control based on least-privilege principle
 - a . RBAC and MFA
5. Implement monitoring controls to identify attacks
 - a . Alerts for utilization or spam access
6. Be knowledgeable of common database vulnerabilities
 - a . Injection, RCE, etc.
7. Plan for disasters
 - a . Have backups and alternatives if a database goes down
8. Do not store all confidential data in the same location/service
 - a . The breach of a single database should not provide enough info to identify

Sources:

<https://www.microsoft.com/en-us/security/blog/2023/07/05/11-best-practices-for-securing-data-in-cloud-services/>

<https://www.ibm.com/think/topics/database-security>

Category 5: API and SSH Security

1. Implement Strong Authentication and Authorization

- a. Enable Two-Factor Authentication
- 2. Encrypt Data in Transit
- 3. Limit Data Exposure
 - a. Ensure that API only return necessary user data
- 4. Direct Usage, no plugins

Category 6: Data Networking Security

- 1. Implement Strong Password Policies
 - a. Should be in the range of a certain length, but not a set number
 - a. Should not be a commonly used password
 - b. Should allow for nearly all characters to be used
 - c. Should not allow for the use of breached passwords
- 2. Use Firewalls and Intrusion Detection System

Category 7: Malware and Bad Software

- 1. Keep Software and Systems Updated
- 2. Regularly BackUp User Data
- 3. Implement Spam and Malware Checks

Category 8: Blockchain and Cryptography

- 1. Use Strong Cryptographic Algorithms
 - a. “Strong cryptography is based on industry-tested and accepted algorithms along with key lengths that provide **a minimum of 112-bits of effective key strength** and proper key-management practices.” - <https://www.pcisecuritystandards.org/glossary/strong-cryptography/>
 - b. Uses hash functions, or symmetric-key algorithms, or asymmetric-key algorithms

4.1 Classes of Cryptographic Algorithms

There are three basic classes of **approved** cryptographic algorithms: hash functions, symmetric-key algorithms and asymmetric-key algorithms. The classes are defined by the number of cryptographic keys that are used in conjunction with the algorithm.

<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57pt1r4.pdf>

- 2. Regularly Update and Patch Systems
- 3. Uses Smart Contract
 - a. [“Less Code, More Security”](#)
 - b. Do not implement temporary fixes, [“A Continuous Vigilance, Not a One-Time Fix”](#)
 - c. [Granular Access Control](#)

Sources:

<https://www.wiz.io/academy/api-security-best-practices>

<https://medium.com/%40chrisbrown798789/7-best-practices-for-secure-api-development-75508a24e4d4>

<https://www.pcisecuritystandards.org/glossary/strong-cryptography/>

<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57pt1r4.pdf>

9. Compliance and Response (may be specific to each category)

1. Define Security Requirements for Software Development (PO.1)
2. Design Software to Meet Security Requirements and Mitigate Security Risks (PW.1)
3. Implement Roles and Responsibilities (PO.2)
4. testing and auditing (Test Executable Code to Identify Vulnerabilities and Verify Compliance with Security Requirements (PW.8))
5. Define and Use Criteria for Software Security Checks (PO.4)
6. integrate compliance at every stage of the SLDC
7. conduct regular risk assessments
8. secure coding practices (Create Source Code by Adhering to Secure Coding Practices (PW.5))
9. third-party risk management (Communicate requirements to all third parties who will provide commercial software components to the organization for reuse by the organization's own software. (PO.1.3))

10. Secure App Development

1. secure coding practices (Create Source Code by Adhering to Secure Coding Practices (PW.5))
 - a. Example 1: Validate all inputs, and validate and properly encode all outputs.
 - b. Example 2: Avoid using unsafe functions and calls.
 - c. Example 3: Detect errors, and handle them gracefully.
 - d. Example 4: Provide logging and tracing capabilities.
2. use secure communication protocols such as HTTPS and TLS
 - a. Example 1: Configure each development endpoint based on approved hardening guides, checklists, etc.; for example, enable FIPS-compliant encryption of all sensitive data at rest and in transit.
3. implement strong authentication and authorization
 - a. use multi-factor authentication
 - b. enforce strong password policies

4. data protection
 - a. use strong encryption algorithms
5. security testing (Test Executable Code to Identify Vulnerabilities and Verify Compliance with Security Requirements (PW.8))
6. Configure the Compilation, Interpreter, and Build Processes to Improve Executable Security (PW.6)
7. Review and/or Analyze Human-Readable Code to Identify Vulnerabilities and Verify Compliance with Security Requirements (PW.7)
8. Test Executable Code to Identify Vulnerabilities and Verify Compliance with Security Requirements (PW.8)
9. Configure Software to Have Secure Settings by Default (PW.9)
10. Identify and Confirm Vulnerabilities on an Ongoing Basis (RV.1)
11. Assess, Prioritize, and Remediate Vulnerabilities (RV.2)
12. Implement and Maintain Secure Environments for Software Development (PO.5)
13. Node.js: keep dependencies updated

11. Cloud Security

1. understand the shared responsibility model: a framework establishing who is responsible for securing different aspects of the cloud-computing environment between the cloud service provider (CSP) and the customer
2. implement identity and access management (IAM)
 - a. implement role-based access control
 - b. use multi-factor authentication
 - c. use principle of least privilege
3. data security
 - a. protect sensitive information by encrypting data at rest and in transit
4. security monitoring and auditing
5. ensure compliance with cloud security policies, regulations, and industry standards
6. develop and regularly test an incident response plan
7. secure containers and containerized workloads
8. Implement and Maintain Secure Environments for Software Development (PO.5)
 - a. PO.5.1: Separate and protect each environment involved in software development.
 - i. Example 1: Use multi-factor, risk-based authentication and conditional access for each environment.
 - ii. Example 2: Use network segmentation and access controls to separate the environments from each other and from

production environments, and to separate components from each other within each non-production environment, in order to reduce attack surfaces and attackers' lateral movement and privilege/access escalation.

- iii. Example 10: Configure and implement measures to secure the environments' hosting infrastructures following a zero trust architecture (zero trust architecture: a security framework that assumes no user or device is inherently trustworthy and requires strict verification and authorization for every access request, regardless of network location)

9. Identify and Confirm Vulnerabilities on an Ongoing Basis (RV.1)

10. Assess, Prioritize, and Remediate Vulnerabilities (RV.2)

11. Define Security Requirements for Software Development (PO.1)

12. Data Privacy and Anonymization

1. Define Security Requirements for Software Development (PO.1)

- a. "This includes requirements from internal sources (e.g., the organization's policies, business objectives, and risk management strategy) and external sources (e.g., applicable laws and regulations)."

2. identify PII

3. map legal requirements

4. implement anonymization techniques

5. monitoring and updates